Project Report: Music Player with Lyrics using REST API

Tarun V

23BTCE284

Full Stack Development

1. Introduction

The *Music Player with Lyrics* project is a web-based application built using **React (Vite)**, **Spotify Web API**, and **Bootstrap** for styling.

The main objective of this project is to provide users with a seamless experience of:

- Searching for songs
- Playing previews
- Viewing lyrics
- Creating and managing playlists

This project combines the functionality of a modern music app with a simple UI and dark theme for better contrast.

2. Objectives

- To implement a music player interface using React.
- To integrate with **Spotify Web API** for music search and previews.
- To use a lyrics API to fetch and display lyrics for selected songs.
- To allow users to create, add, and remove songs from playlists.
- To design the UI with a **dark theme** for better readability.

3. Tools & Technologies

- **Frontend Framework:** React (Vite)
- **Styling:** Bootstrap 5, Custom CSS (Dark Theme)
- **APIs:**
  - Spotify Web API (for search and preview playback)
  - Lyrics API (e.g., lyrics.ovh or Musixmatch API)
- **IDE:** Visual Studio Code
- **Language:** JavaScript (React JSX)

4. System Requirements

- **Software:** Node.js (LTS version), npm, VS Code
- **Browser:** Chrome / Edge / Firefox
- **Dependencies Installed:**
  - react, react-dom
  - vite
  - bootstrap

- axios (for API calls)

5. Project Structure

```
music-player-lyrics/
│── index.html
│── package.json
│── vite.config.js
│
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   ├── App.css
│   ├── components/
│   │   ├── Player.jsx
│   │   ├── Playlist.jsx
│   │   └── Lyrics.jsx
│
└── node_modules/
```

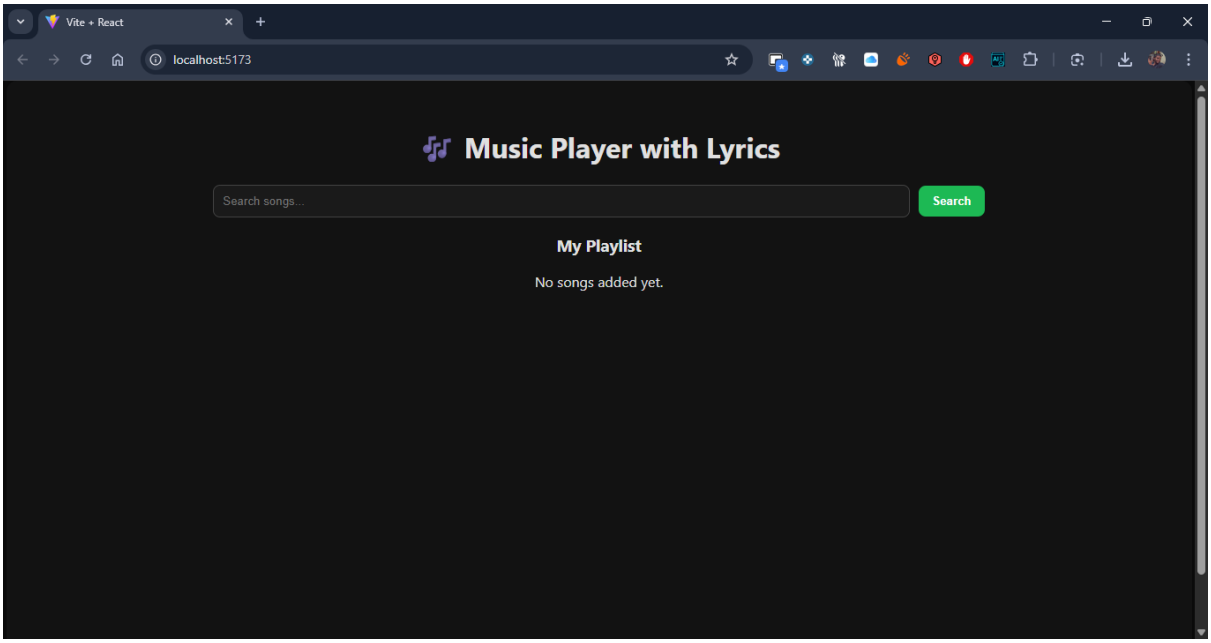
6. Implementation Details

6.1 Authentication

- Users authenticate with **Spotify API** using a Redirect URI.
- A temporary access token is generated for searching and previewing songs.

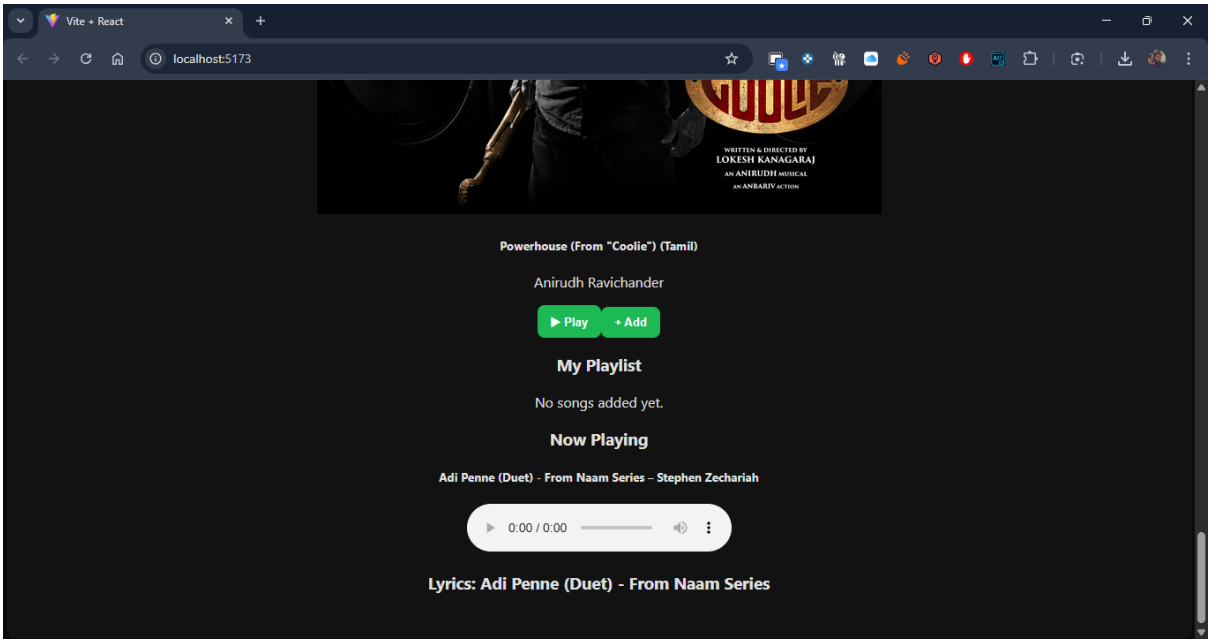
6.2 Search & Player Component

- Users can search for songs or artists.
- Search results display **thumbnail, song name, and artist name** in a **grid view (library style)**.
- A play preview button allows playback of 30-second samples.



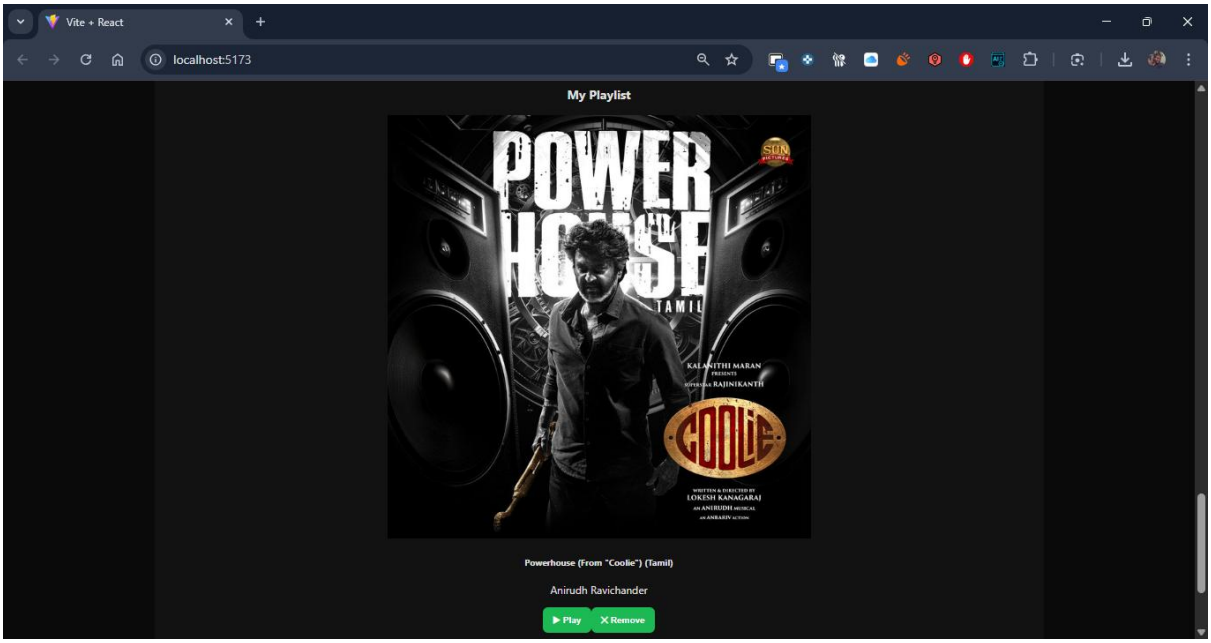
6.3 Lyrics Component

- When a user selects a track, lyrics are fetched via an external Lyrics API.
- Lyrics are displayed below the player with proper formatting.



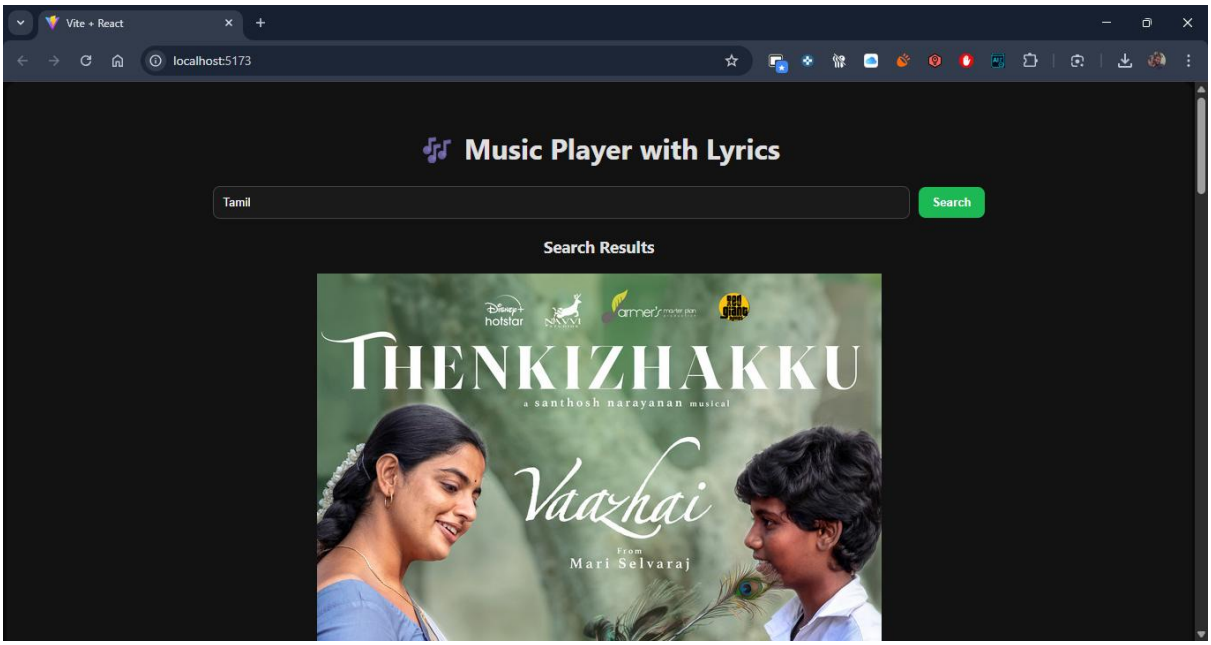
6.4 Playlist Component

- Users can add/remove songs from a playlist.
- The playlist is displayed in a **card/grid format** with song thumbnails.
- Each track in the playlist has a "Remove" button.



7. User Interface

- The application uses a **dark theme** (background: #121212; color: #fff;).
- Buttons and search bar styled with Bootstrap (btn-dark, btn-primary).
- Layout divided into **3 main sections**:
    1. Search Results (library style grid)
    2. Lyrics Viewer
    3. Playlist Manager

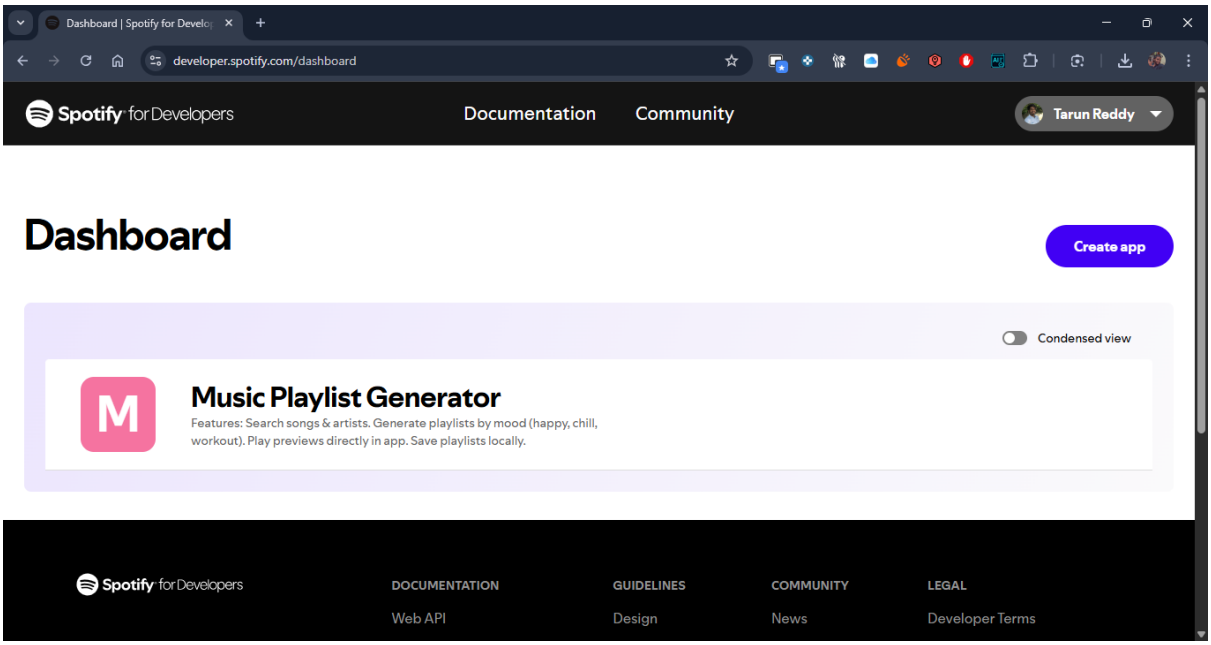


8. Challenges Faced

- Handling **CORS issues** while calling external APIs.
- Managing **Spotify token expiry**.
- Designing a **responsive layout** for both desktop and mobile screens.
- Styling the search results as a grid (library view) instead of list view.

9. Future Enhancements

- Add **full track playback** using Spotify Web Playback SDK.
- Implement **user authentication** (Spotify login).
- Allow saving playlists to **Spotify user account**.
- Add **mood-based playlist generation** (Happy, Chill, Workout).

10. Conclusion

The *Music Player with Lyrics* project successfully demonstrates the integration of multiple APIs within a React app. It provides users with an interactive music experience, combining **search, play, lyrics, and playlist management** in one place.